LABORATORY ACTIVITY 9

Secure authentication

Edgardo Gabriel L. Paclibar

4/BSCS/A

PHP Web Security and Secure User Authentication

campus. Laboratory Inventory System

13 landscape screenshots

Registration and validation; stored password hashes and database structure.

Secure login, protected dashboard, session logout and restricted access.

HTML and SQL-injection-style input rejection.

Security testing and completed worksheet

Parts A-E of the activity are documented. The report includes the recorded 45-check security result, observations, and answers to all four guided questions.

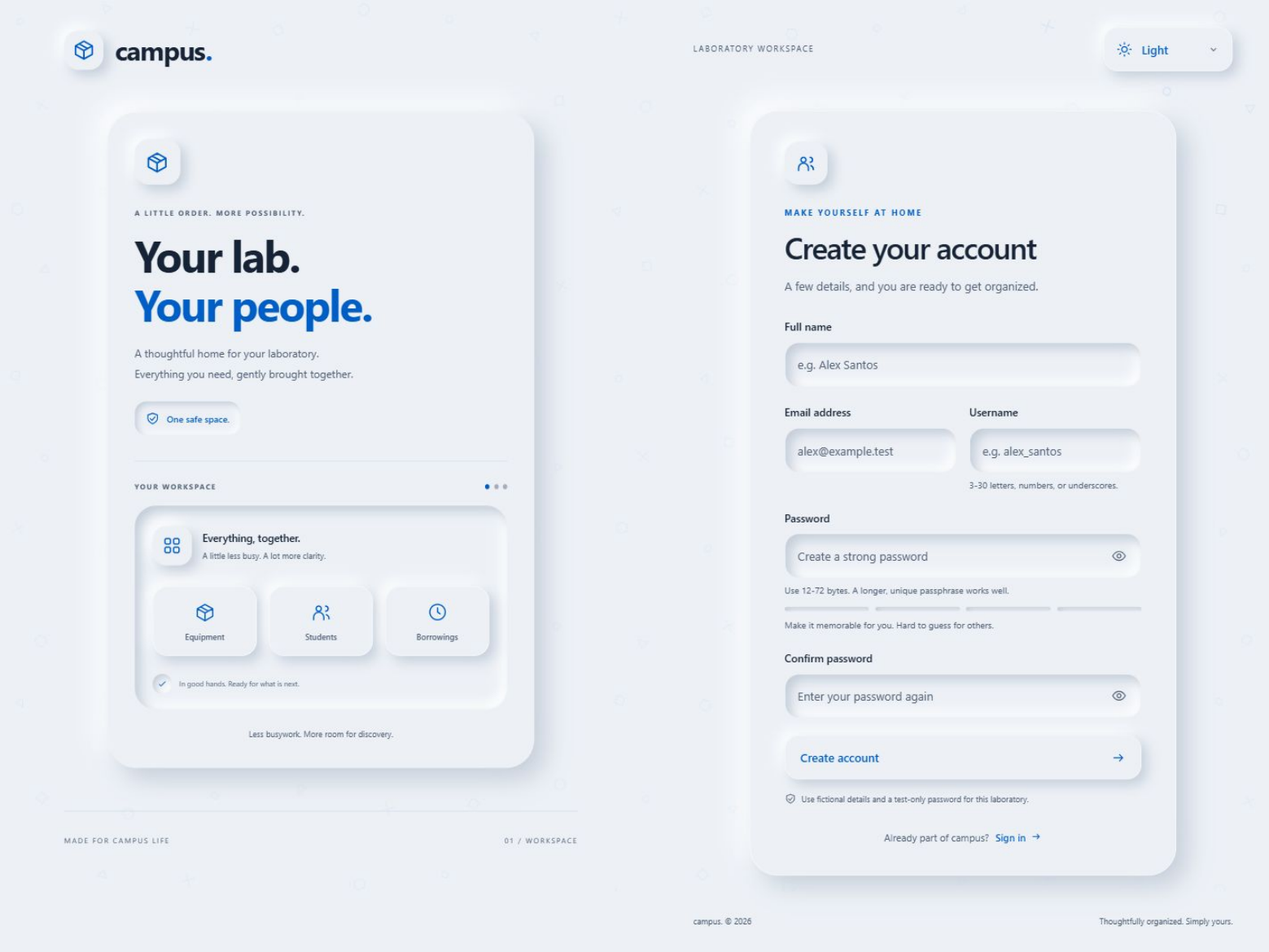
Learning outcomes

Identify common web vulnerabilities and implement password hashing and secure authentication techniques.

Requirement source: supplied Laboratory Activity 9, ICT 07, textbook pages 40-42.
Evidence date: 4 November 2026. Fictional accounts only; passwords are excluded.
Application captures use the current interface. Database captures show a removed test record.

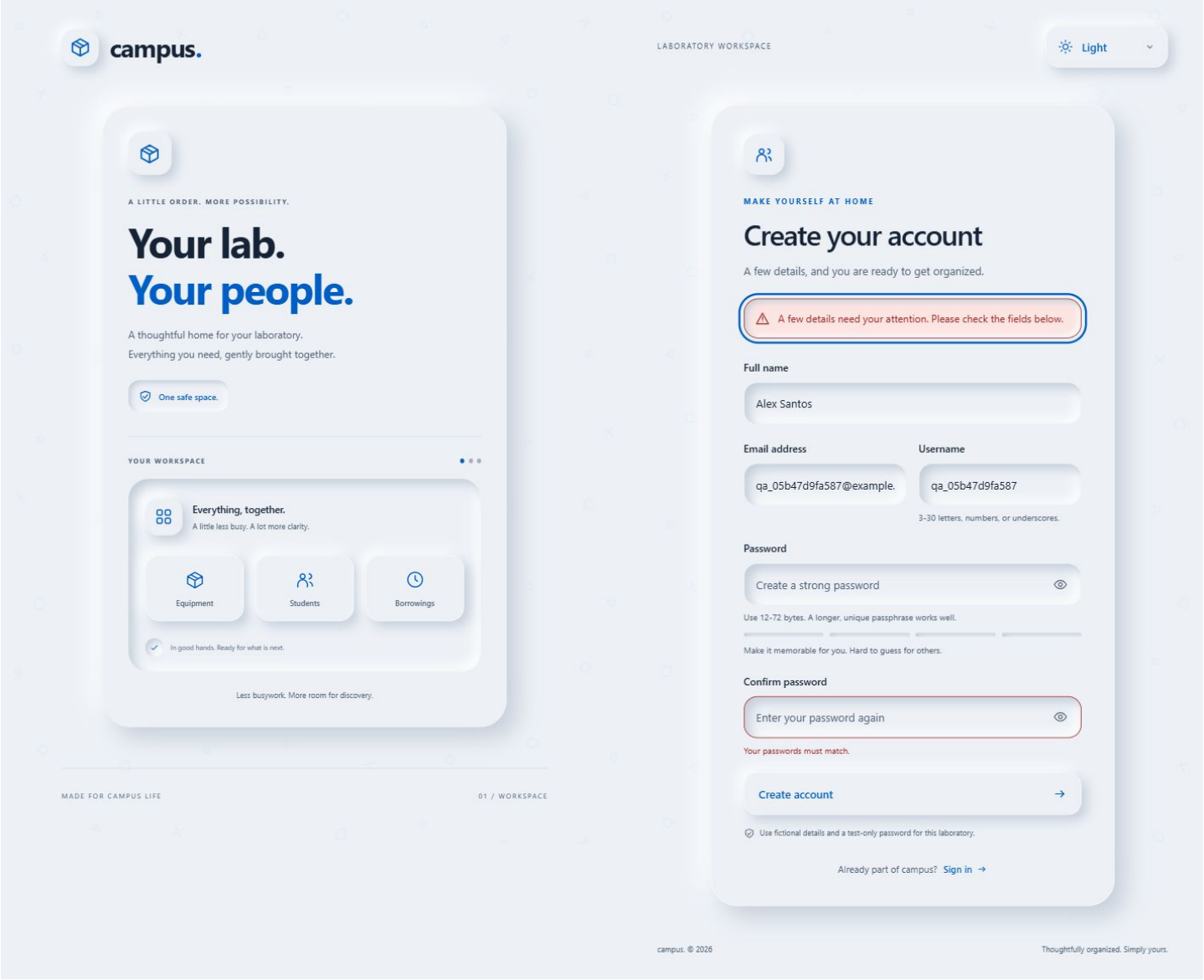
Secure registration form

Part A - Full Name, Email Address, Username, Password and Confirm Password are present.



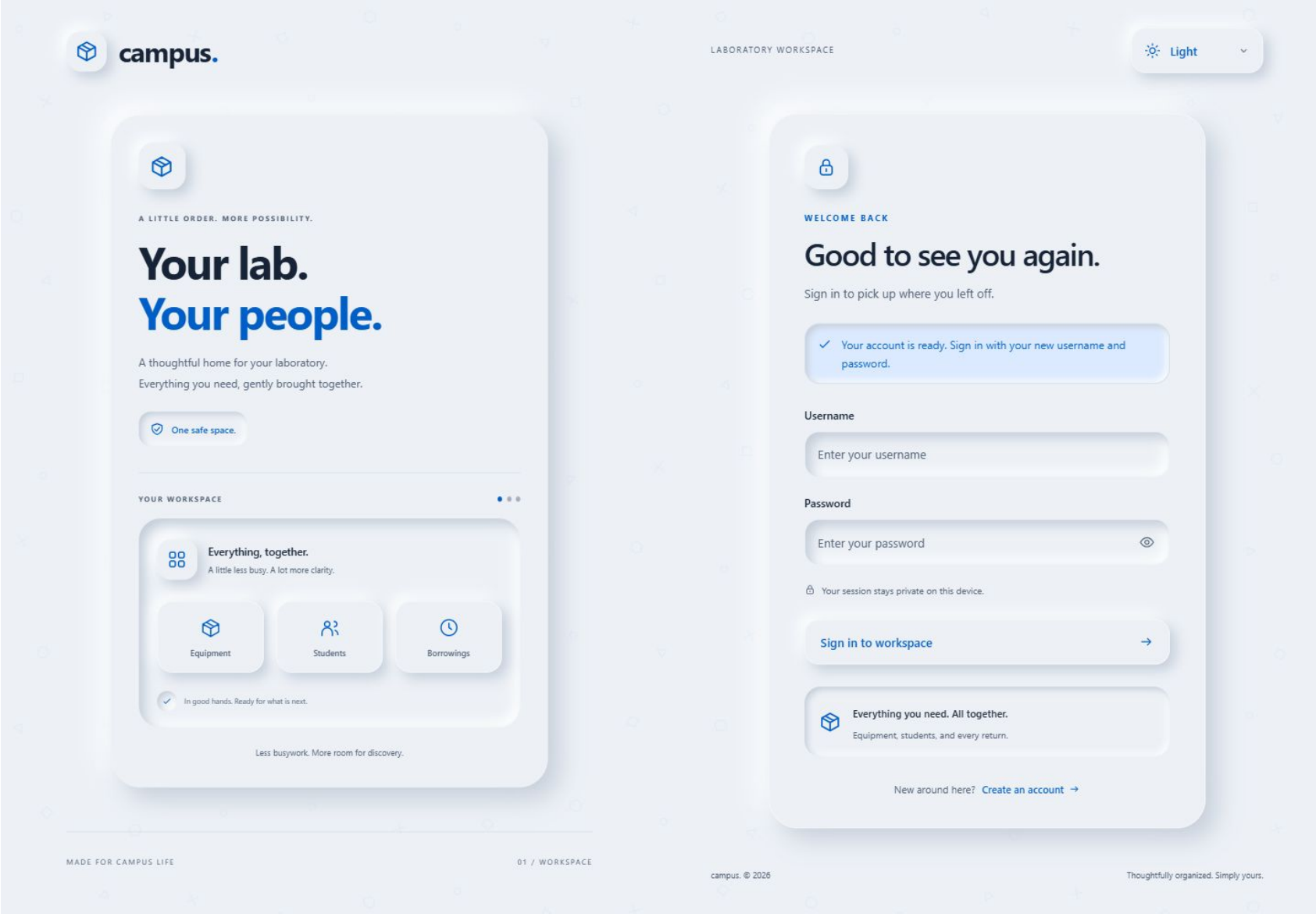
Password mismatch validation

Part A - Unequal passwords are rejected and the confirmation field displays an error.



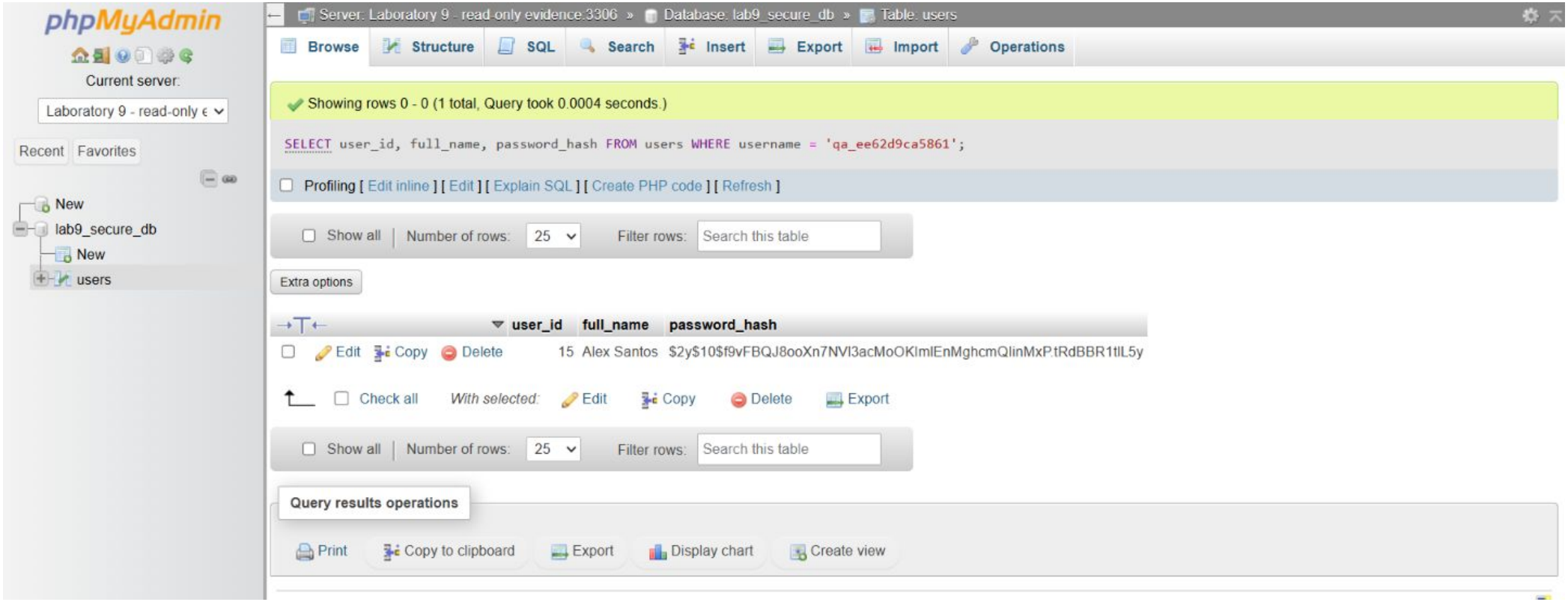
Successful registration / login form

Parts A and C - A valid fictional account redirects to login with an account-ready confirmation.



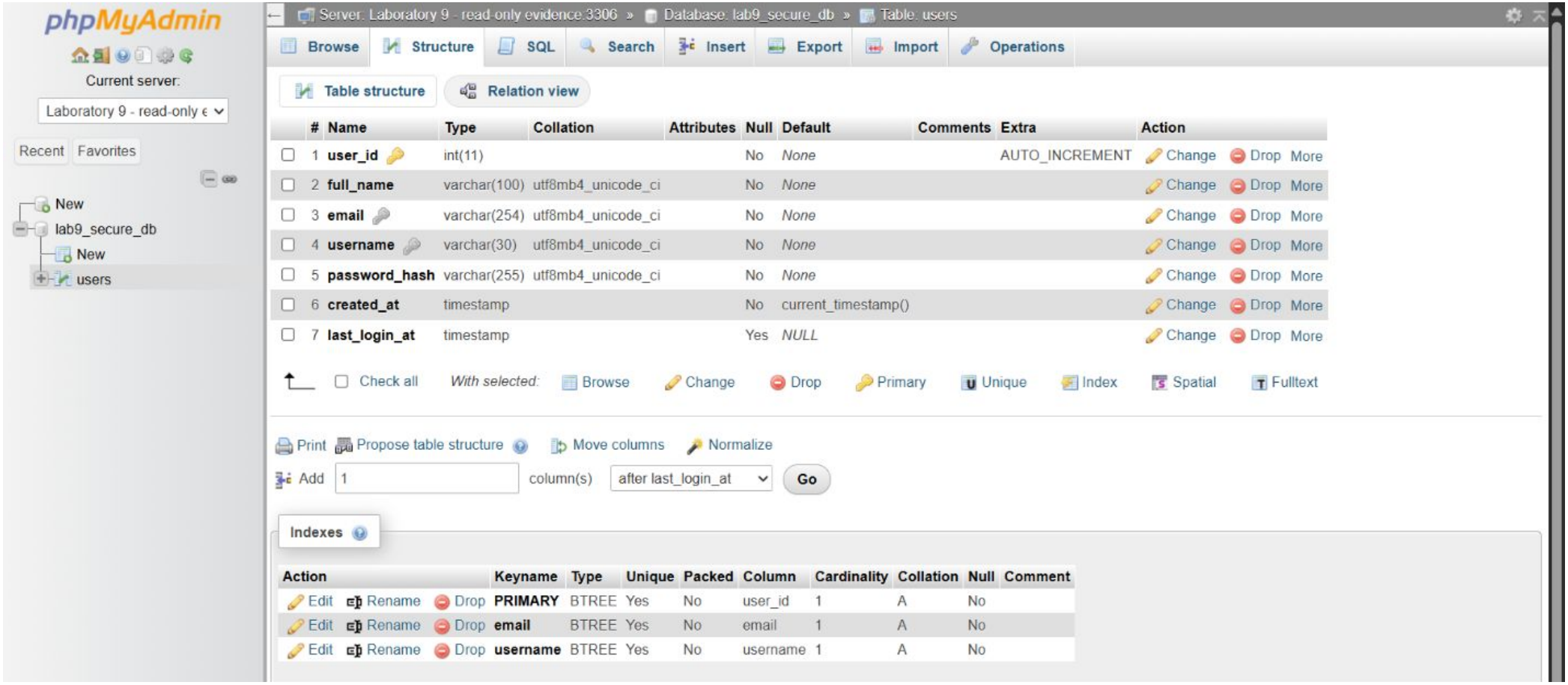
Stored password hash in phpMyAdmin

Part B - A filtered query shows password_hash for a disposable fictional account; the entered password is not stored.



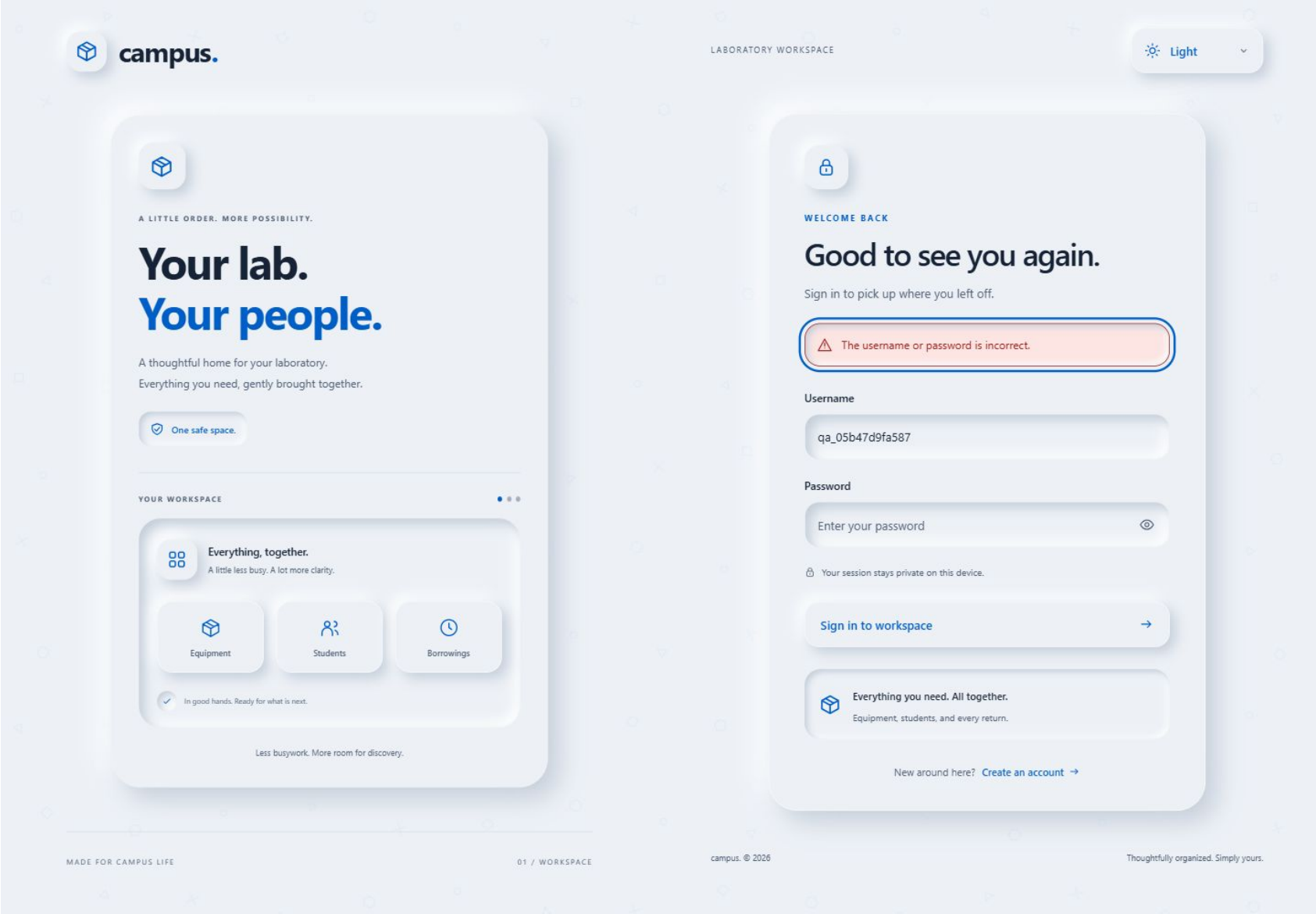
Authentication table structure

Part B - users contains password_hash VARCHAR(255), identity fields and login timestamps; no plaintext password column.



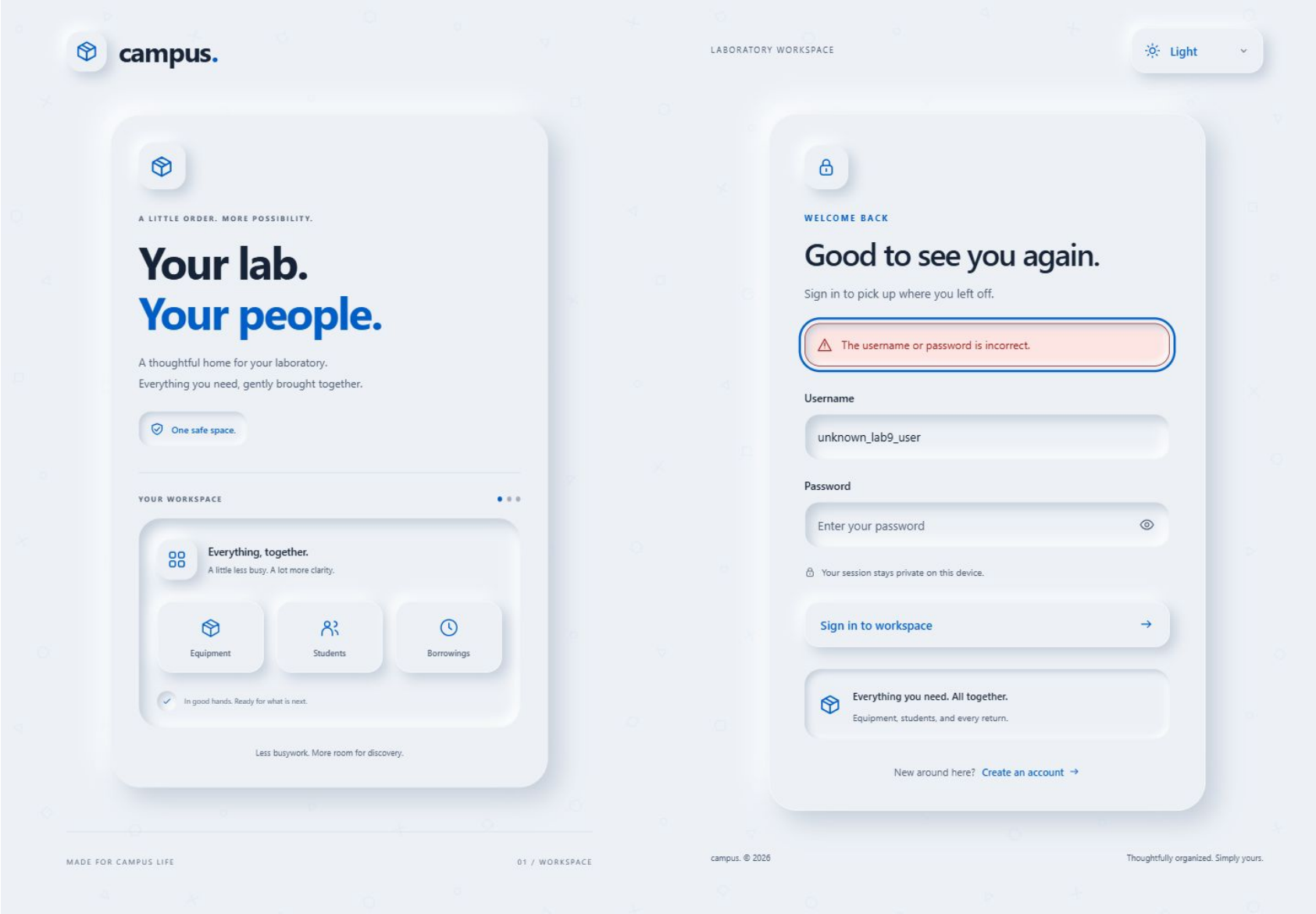
Incorrect password rejected

Parts C and E - A wrong password returns the generic error without exposing SQL or sensitive system information.



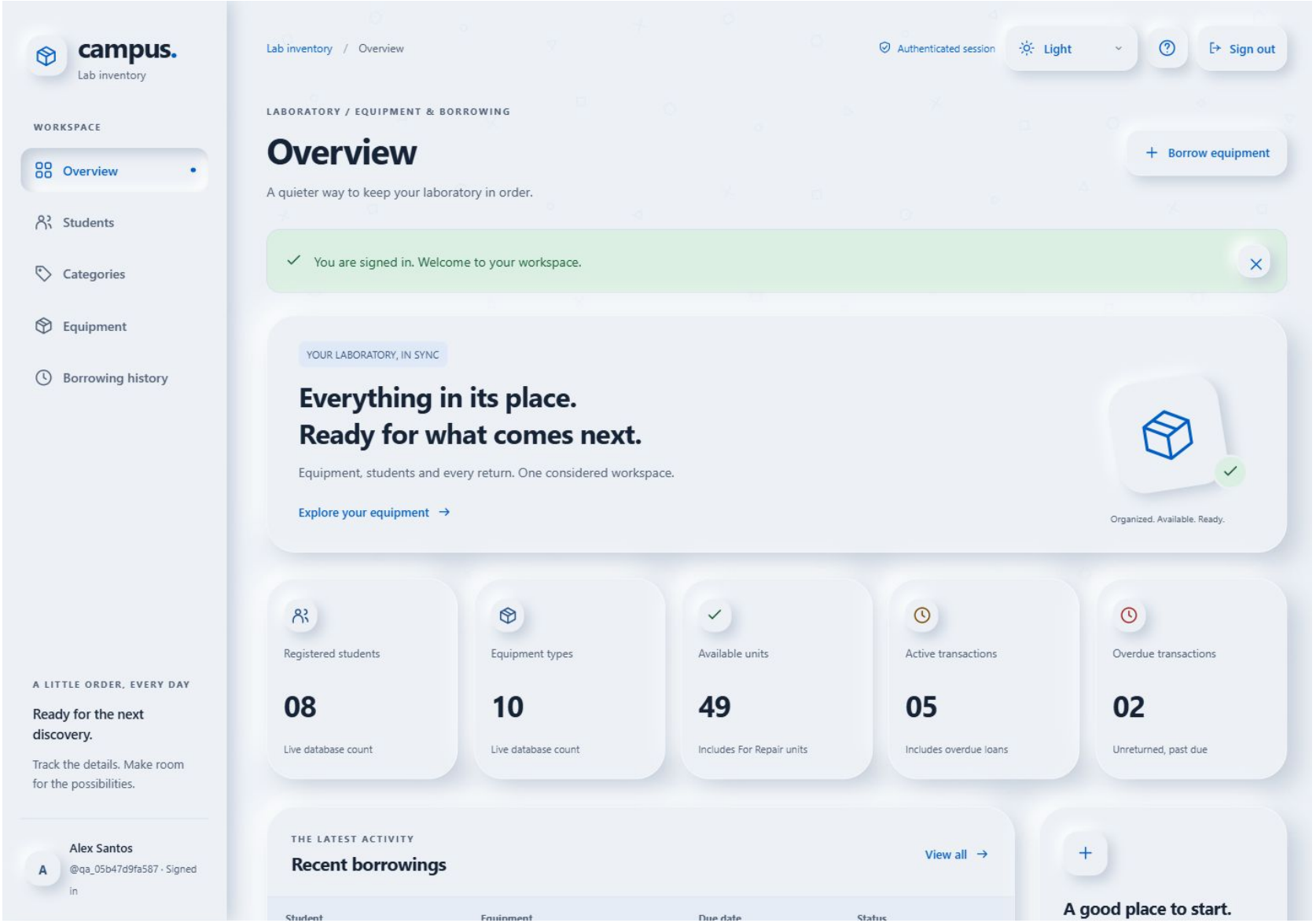
Invalid username rejected

Part E - An unknown username returns the same generic login failure as an incorrect password.



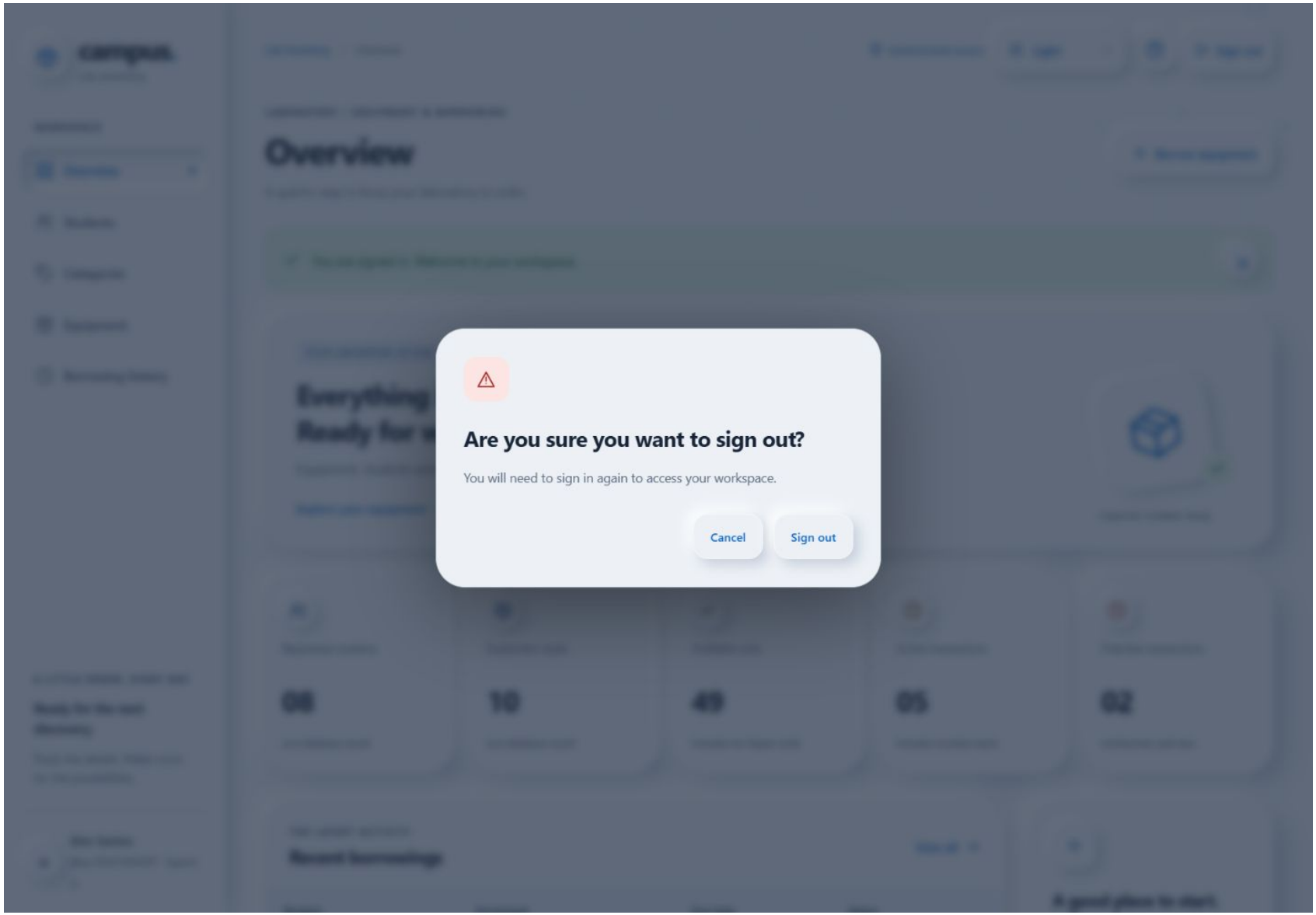
Authenticated dashboard

Parts C and D - Correct login opens dashboard.php and displays the fictional account identity and live counts (top desktop view).



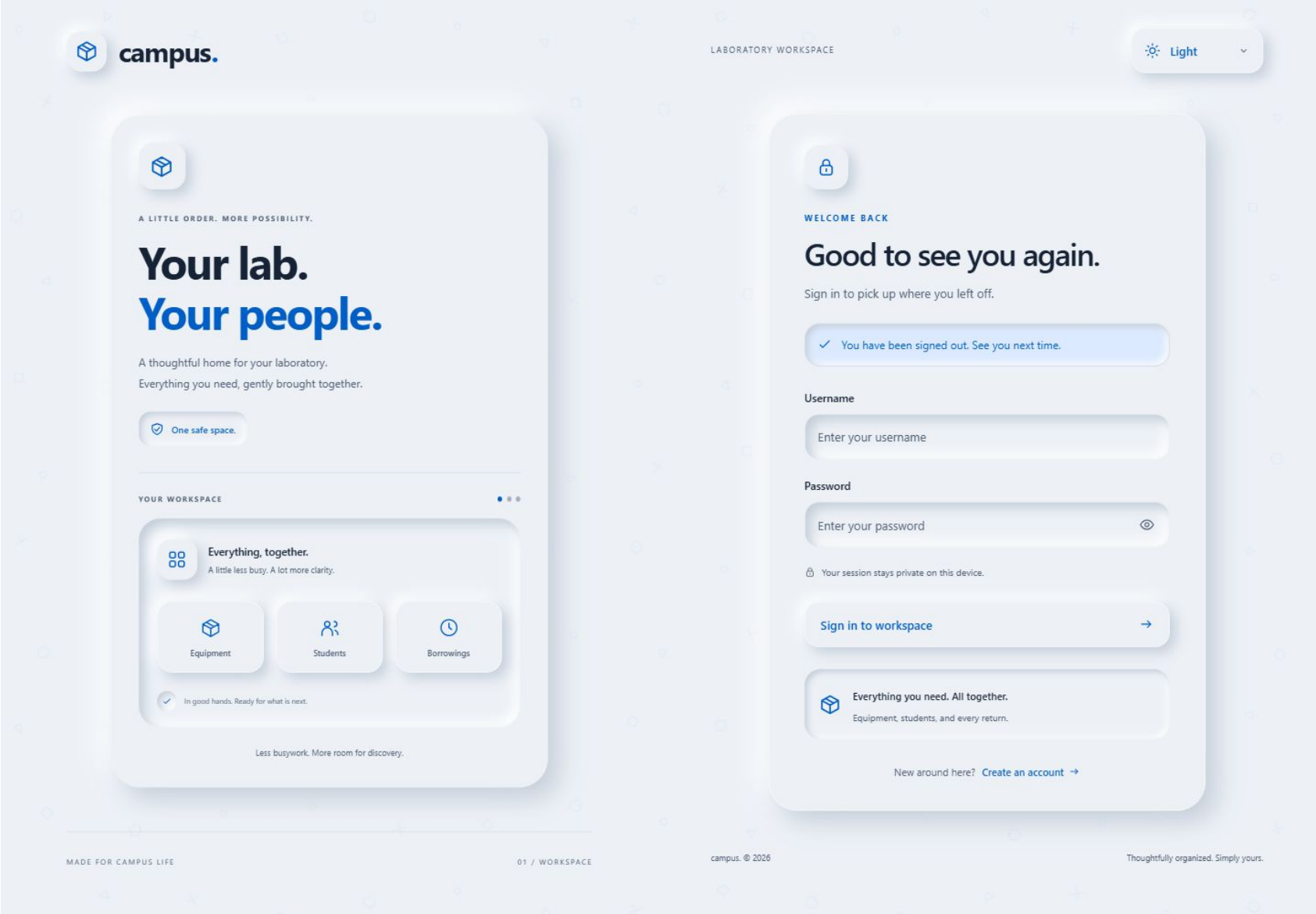
Sign-out confirmation

Part D - Cancel preserves the session. Confirming Sign out submits the CSRF-protected logout request.



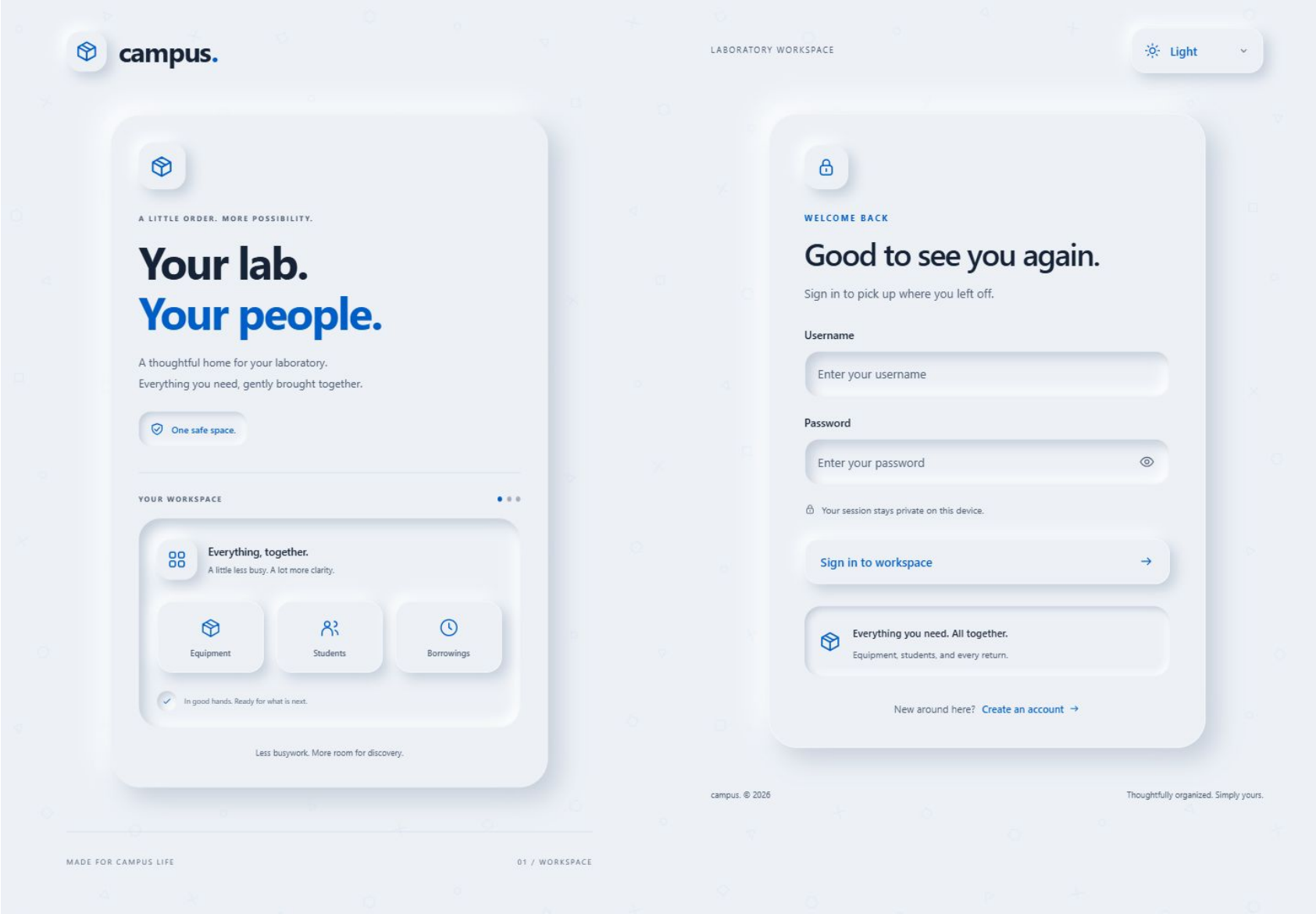
Successful sign-out

Part D - After clearing session data, expiring its cookie and calling session_destroy(), the app returns to login.



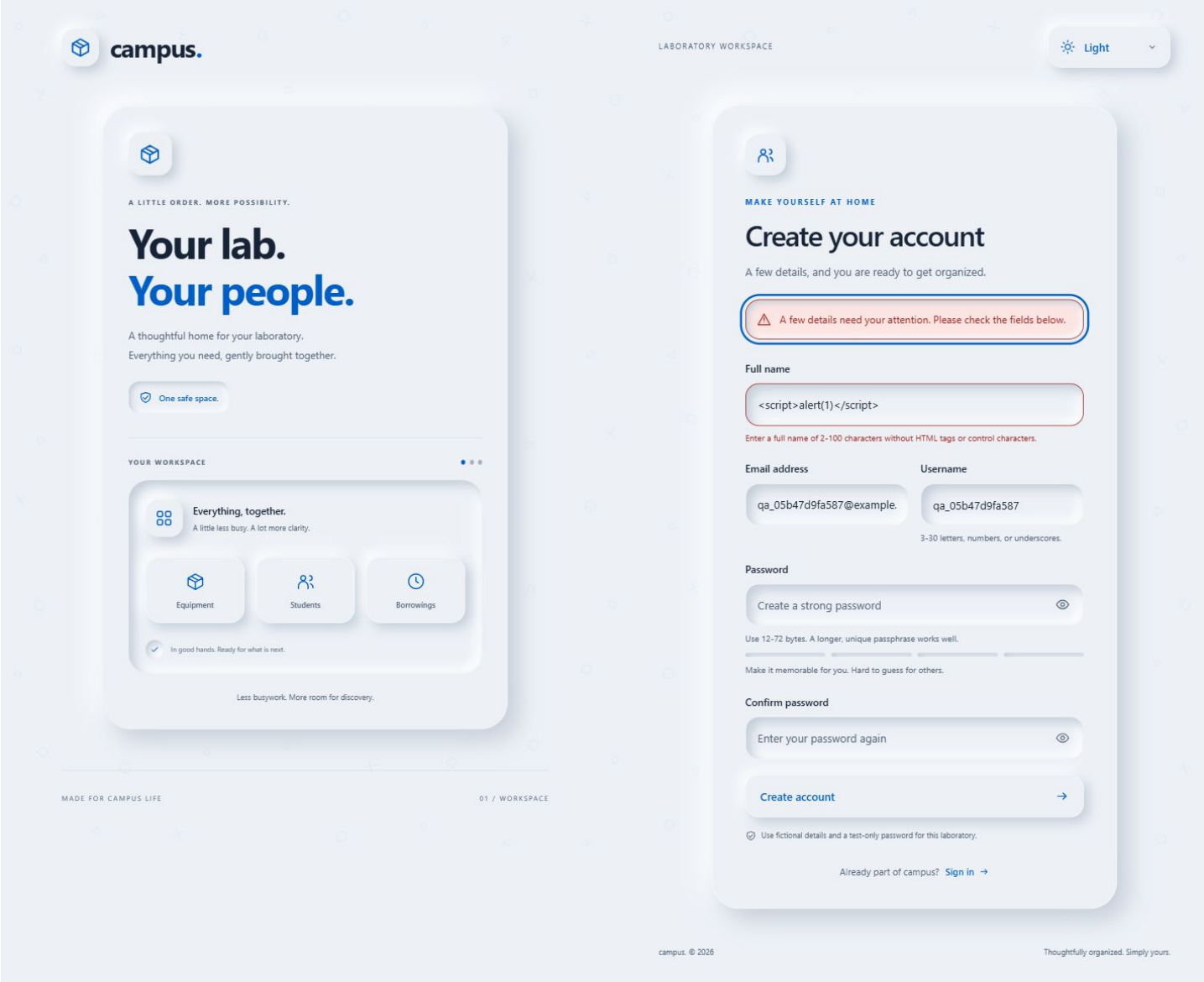
Protected page redirects a guest

Part D - Requesting /Laboratory9-SecureAuth/dashboard.php after logout redirected to /Laboratory9-SecureAuth/login.php.



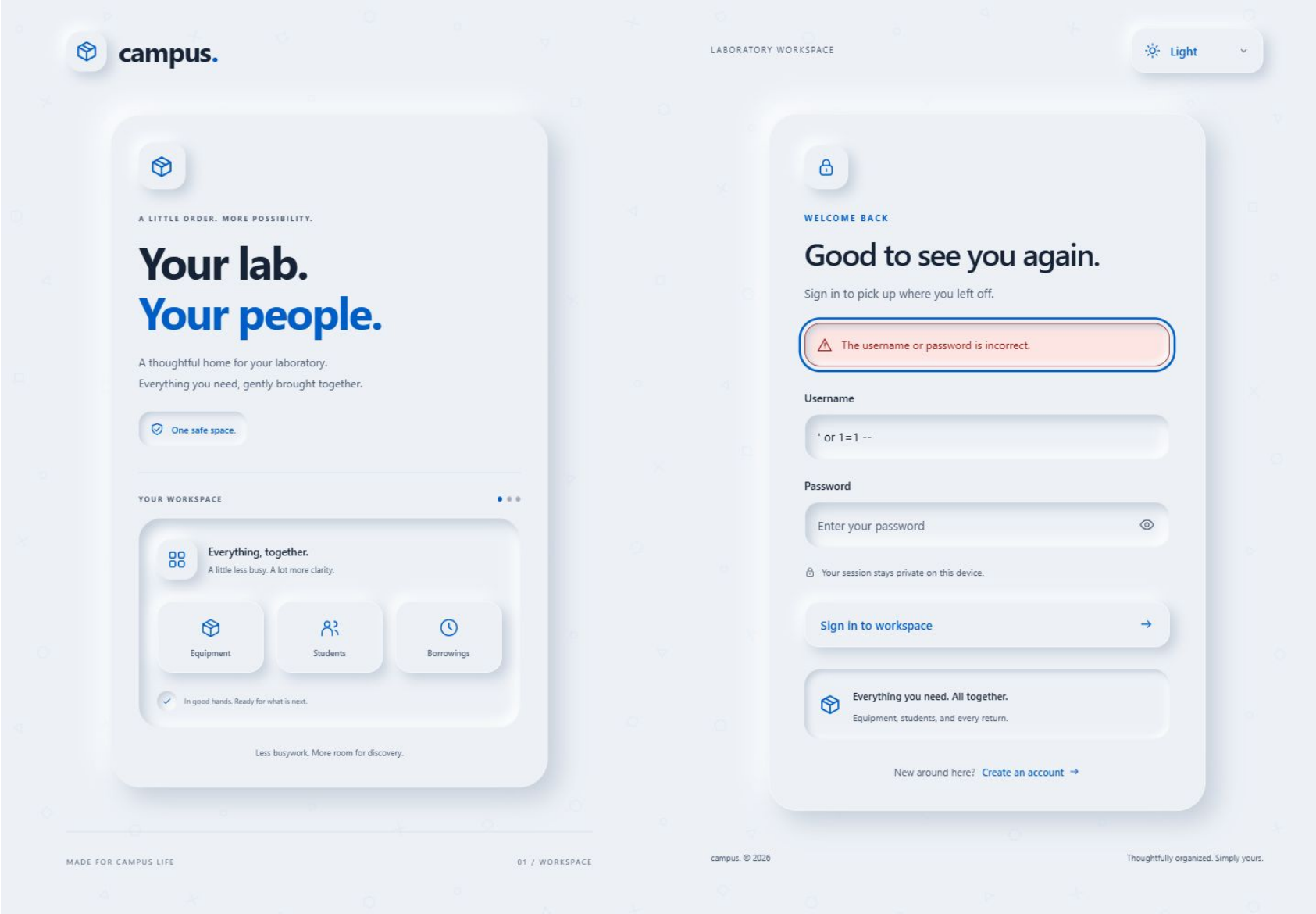
HTML input rejected

Part E - The full-name input <script>alert(1)</script> was rejected and escaped; no JavaScript dialog appeared.



SQL-injection-style login rejected

Part E - The username input ' OR 1=1 -- did not authenticate. The application returned its generic login error.



Security tests and observations

Part E - Recorded HTTP/database verification on 4 November 2026: 45 passed, 0 failed; completed=true.

Test area	Observed result
Valid registration	Inserted an account and redirected to login with a success notice.
Invalid fields / mismatch	Required fields, email syntax, username rules, password length and matching checks rejected invalid input.
Password storage	A recognized hash verified against the candidate password and differed from plaintext. Equal passwords produced distinct salted hashes.
Correct login	Rotated the session ID, stored the authenticated username, updated last_login_at and opened the protected dashboard.
Incorrect password / unknown user	Both rejected using the same generic message.
Malicious input	HTML and injection-style registration/login values were rejected. Reflected HTML was escaped; inventory tables remained intact.
Guest access / logout	All 10 tested protected entry points redirected guests. Logout expired the cookie; replaying the old session did not restore access.
Session expiry	Simulated 30-minute idle and 8-hour absolute expiry both forced login.
CSRF / attempt limits	Invalid CSRF rejected. Five failed logins triggered HTTP 429; a new cookie session did not reset the limit.
Internal resources / cleanup	Four internal-resource requests returned 403. Disposable test accounts were removed.

Environment: Apache 2.4.58, PHP 8.2.12, MariaDB 12.3.2 and phpMyAdmin 5.2.1. Results are saved in SECURITY_TEST_RESULTS.json. The 13 screenshot pages document the same implementation; the database images were captured earlier on the same date. These local checks are not a penetration-test certification.

Guided questions

Completed worksheet - the four questions in the supplied Laboratory Activity 9.

1. What are some common security vulnerabilities in web applications?

Common vulnerabilities include SQL injection, cross-site scripting (XSS), cross-site request forgery (CSRF), weak password storage, session fixation, and missing access checks. Prepared statements separate SQL from untrusted values. Context-appropriate output escaping prevents user content from becoming executable markup. Session-bound CSRF tokens protect changes, while session ID rotation and authentication checks protect restricted pages.

2. Why should passwords never be stored as plain text?

Anyone who obtains a plaintext password table immediately learns the passwords. Reuse can expose accounts on other services as well. Salted, computationally expensive password hashes allow verification without keeping the original password. Hashing reduces the impact of a database leak, but it does not remove the need for strong passwords and database access controls: attackers can still try to guess passwords against leaked hashes.

3. What is password hashing, and how does it improve security?

Password hashing applies a one-way, password-specific algorithm to produce a verification value. PHP password_hash() automatically generates a salt, so equal passwords normally produce different stored values. Its adjustable computational cost slows guessing. This project uses PASSWORD_DEFAULT with a 255-character database column. Hashing is not reversible encryption; the application checks a candidate password instead of decrypting a stored value.

4. What is the difference between password_hash() and password_verify()?

password_hash() creates the value stored during registration. password_verify($candidate, $storedHash) returns true or false during login. The stored hash includes the algorithm, salt, and cost needed for verification. The application must not generate a new salted hash during login and compare the two hash strings, because independently generated salts can differ.

Implementation and laboratory rules

Project file mapping, essential security logic and safe local demonstration.

Activity	Implementation
Part A - Registration	register.php; register_account() in includes/auth.php.
Part B - Hashing	password_hash($password, PASSWORD_DEFAULT); users.password_hash VARCHAR(255).
Part C - Login	login.php; prepared lookup and password_verify().
Part D - Sessions	require_auth(); session_regenerate_id(true); dashboard.php; logout.php and session_destroy().
Part E - Tests	bin/test_auth.py; SECURITY_TEST_RESULTS.json; screenshot evidence and observations.

Essential PHP logic

```
$hash = password_hash($password, PASSWORD_DEFAULT);
// Stored using an INSERT with bound parameters.
$verified = password_verify($password, $storedHash);
session_regenerate_id(true);
$_SESSION['username'] = $user['username'];
// Protected pages require_auth(); logout calls session_destroy().
```

Run the project

Place Laboratory9-SecureAuth in XAMPP htdocs. Import sql/lab9_secure_db.sql and sql/auth.sql into a fresh local database. Start the required services and open http://localhost/Laboratory9-SecureAuth/register.php. Register fictional details, sign in, inspect the dashboard, then sign out.

Safety rules applied

Only disposable fictional accounts were used for evidence. Passwords, database credentials, session cookies and real account rows are excluded. Inputs are validated, output is escaped, and user-dependent SQL uses prepared statements. Backups were retained. Test sessions were signed out and temporary test records were removed. The original phpMyAdmin configuration was restored after database capture.

After the final demonstration, stop Apache and MySQL when no other projects need them. Existing services remained available for the active workspace during report preparation.